

Production Incident Report: Asynchronous Checkout Processing Failures

Question 1

When did the problem start? Include the timestamp and the evidence you used.

Conclusion

The incident began on **2026-07-02 at 14:32:40.073**, defined by the receipt of the earliest checkout request that subsequently failed background execution.

Evidence

- **Boundary Timeline:** Below is the chronological sequence of checkout requests immediately preceding and following the failure boundary:

Web Request Timestamp	Request ID	User ID	Parity	Background Outcome
2026-07-02 14:32:04.318	527756195a1ceb9c	52214	Even	Completed
2026-07-02 14:32:19.167	7a23b65c86745ba6	75567	Odd	Completed
2026-07-02 14:32:31.237	ca1681fe16167f60	95820	Even	Completed
2026-07-02 14:32:40.073	16ce72300cf58a32	59787	Odd	Failed
2026-07-02 14:32:40.206	f0285d88709fce1b	57034	Even	Completed
2026-07-02 14:32:42.654	2c60cb6a51a7a14a	91486	Even	Completed

- **Earliest Background Worker Failure:** The background worker logged the exception at **14:32:42.692**:

```
2026-07-02 14:32:42.692 ERROR [worker] upstream call failed
request_id=16ce72300cf58a32 err=ECONNRESET upstream=10.0.3.44:8443 (retries
exhausted)
```

Reasoning

Although the asynchronous worker logged the connection failure at **14:32:42.692**, the transaction lifecycle began when the HTTP server received the request at **14:32:40.073**. Prior checkout transactions, including those for odd-numbered users (such as `user_id=75567` at **14:32:19.167**), succeeded. The request at **14:32:40.073** represents the earliest transaction in the logs that failed to process.

Question 2

Which endpoint is affected? Support your answer with evidence from the logs.

Conclusion

The incident is isolated exclusively to the **POST /checkout** endpoint. All other synchronous and asynchronous endpoints performed normally throughout the 24-hour log period.

Evidence

- **HTTP Server Request Metrics (24-Hour Span):** The following table summarizes the HTTP status code counts for all endpoints in `web.log` (parameterized product detail paths such as `/product/21367` are normalized to `/product/:id`). The rows sum to all 100,000 request lines in the file:

Path	HTTP GET	HTTP POST	Status 200	Status 202	Status 401	Status 404	Total
<code>/products</code>	25,990	0	25,990	0	0	0	25,990
<code>/product/:id</code>	15,076	0	14,884	0	0	192	15,076
<code>/cart</code>	0	12,037	12,037	0	0	0	12,037
<code>/search</code>	11,009	0	11,009	0	0	0	11,009
<code>/login</code>	0	10,044	0	10,044	0	0	10,044
<code>/checkout</code>	0	9,875	0	9,875	0	0	9,875
<code>/orders</code>	0	7,998	0	7,998	0	0	7,998
<code>/api/user</code>	7,971	0	7,853	0	118	0	7,971

- **Non-2xx Responses Investigated and Excluded:** The 192 **404** responses on `/product/:id` (1.27% of product detail requests) and the 118 **401** responses on `/api/user` (1.48% of those requests) occur throughout the 24-hour window, beginning hours before the incident (first **404** at `03:54:29`, first **401** at `02:24:48`), and their hourly counts track the overall traffic curve with no step change at the `14:32:40.073` boundary. They are consistent with routine traffic (requests for nonexistent product IDs and expired or unauthenticated sessions) and were excluded from the incident scope.
- **Asynchronous Queue Execution Outcomes:** The HTTP server returns status code **202** (Accepted) for `/login`, `/orders`, and `/checkout` endpoints, offloading processing to the background worker. The table below correlates these requests with the background outcomes in `worker.log`:

Originating Path	Queued Background Tasks	Completed Tasks	Failed Tasks	Failure Rate
<code>/login</code>	10,044	10,044	0	0.00%
<code>/orders</code>	7,998	7,998	0	0.00%
<code>/checkout</code>	9,875	7,490	2,385	24.15%

Reasoning

Every transaction routed to `/checkout` returned a status code **202** to the client. This means the client-facing application perceived the order submission as successful. However, the background worker failed to process

2,385 of these tasks. Because other asynchronous routes (`/login` and `/orders`) had a 100% success rate, the scope of background failures is confirmed to be isolated to the checkout flow.

Question 3

What do the failing requests have in common? Identify the pattern and support it with numbers.

Conclusion

Failing requests are exclusively checkouts submitted by users with odd user IDs, failing due to TCP connection resets (`ECONNRESET`) encountered by the background worker during outbound calls to the upstream host `10.0.3.44:8443`.

Evidence

- Failed Task Error Profile:** All 2,385 failed tasks share the identical signature shown below:

Log Attribute	Value
Path	<code>/checkout</code>
User ID Parity	Odd (<code>user_id % 2 != 0</code>)
Log Level	<code>ERROR</code>
Thread Name	<code>[worker]</code>
Error Signature	<code>err=ECONNRESET</code>
Destination Upstream	<code>10.0.3.44:8443</code>
Final Status	<code>(retries exhausted)</code>

- Post-Incident Checkout Metrics (14:32:40.073 to 23:59:53.001):** The breakdown of checkout tasks initiated during the incident window by user ID parity:

Segment	Total Attempts	Completed Tasks	Failed Tasks	Failure Rate
Even User IDs	2,886	2,886	0	0.00%
Odd User IDs	2,741	356	2,385	87.01%

Reasoning

The dataset shows a strict divide along user ID parity starting at `14:32:40.073`. Even user IDs experienced no failures, while odd user IDs experienced an 87.01% failure rate. The fact that 12.99% (356) of odd user checkouts succeeded during this period indicates that the upstream service `10.0.3.44:8443` was not completely unreachable, but was resetting connections intermittently.

Question 4

How many distinct users were affected?

Conclusion

Exactly **2,335 distinct users** were affected by the checkout processing failures.

Evidence

- **Affected User Breakdown:**

Parameter	Count
Total Failed Checkout Tasks	2,385
Unique Affected User IDs	2,335
Users with Exactly 1 Failed Attempt	2,286
Users with Exactly 2 Failed Attempts	48
Users with Exactly 3 Failed Attempts	1

The distribution is computed directly by `analyze_logs.py` and reconciles exactly: $2,286 + (48 \times 2) + (1 \times 3) = 2,385$ failed tasks across 2,335 users. The single user with three failed attempts is `user_id=38397`, whose checkout requests were received at `14:36:41.336`, `15:46:58.037`, and `18:55:20.864`.

- **Sample Log Traces for a User with Multiple Attempts (User 85171 — one failure, then one successful retry):**

- First attempt at `16:20:01.325` (Failed in worker at `16:20:03.046`):

```
web.log:    2026-07-02 16:20:01.325 INFO [request] method=POST
path=/checkout status=202 latency_ms=57 user_id=85171
request_id=598c06abdaf274a1
worker.log: 2026-07-02 16:20:03.046 ERROR [worker] upstream call failed
request_id=598c06abdaf274a1 err=ECONNRESET upstream=10.0.3.44:8443
(retries exhausted)
```

- Second attempt at `16:41:44.904` (Succeeded in worker at `16:41:45.637`):

```
web.log:    2026-07-02 16:41:44.904 INFO [request] method=POST
path=/checkout status=202 latency_ms=86 user_id=85171
request_id=c454574bd7a6f557
worker.log: 2026-07-02 16:41:45.637 INFO [worker] job completed
request_id=c454574bd7a6f557 duration_ms=557
```

Reasoning

Extracting the `user_id` values from the 2,385 failed checkout requests and filtering for unique values yields 2,335 unique users. The difference of 50 between failed tasks and unique users is explained by 49 users with

repeated failures: 48 users failed twice and one user (38397) failed three times, contributing 50 extra failure records. None of these 49 multi-failure users ever had a completed checkout in the log window. Other users (such as User 85171) had one failed attempt followed by a successful retry due to the intermittent nature of the upstream resets.

Bonus Question

Observed Facts

- All 2,385 worker failures occurred when trying to connect to 10.0.3.44:8443 with `err=ECONNRESET`.
- The failures only affected transactions for users with odd user IDs; even user IDs experienced no failures.
- The connection resets were intermittent, as 12.99% (356) of odd user checkouts post-incident succeeded.
- Average checkout job durations before the incident were similar for both groups (odd: ~967 ms, even: ~956 ms).
- The log files contain 793 instances of `AnalyticsUploadTimeout` errors from the `[metrics-worker]` thread attempting to reach `analytics.internal:9092` starting at 00:00:21.000. These are isolated from the `[worker]` thread.
- `web.log` contains two deployment events. Release v2.14.3 was deployed **87 seconds before** the first failed checkout was received (14:31:13.000 vs. 14:32:40.073).

Evidence

- Deployment events recorded in `web.log`:

```
2026-07-02 06:01:47.000 INFO [deploy] release v2.14.2 deployed to production
(commit 4f756eb)
2026-07-02 14:31:13.000 INFO [deploy] release v2.14.3 deployed to production
(commit 219c39f)
```

- The earlier release v2.14.2 (deployed at 06:01:47.000) was followed by more than eight hours of fully healthy checkout processing, serving as a natural control: deployments alone did not cause failures until v2.14.3.
- The first failed checkout (`request_id=16ce72300cf58a32`) arrived at 14:32:40.073, within two minutes of the v2.14.3 deployment, and no checkout received before that deployment failed.

Reasoning

- The partitioning of traffic based on user ID parity (even vs. odd) suggests a routing split. This is typical for canary deployments, A/B testing, or sharded load balancing.
- Since even user ID checkouts remained fully operational, they either bypassed 10.0.3.44:8443 or called a separate, healthy upstream server.
- The `ECONNRESET` errors indicate TCP connection resets. Because some odd user checkouts succeeded post-incident (12.99%), the destination port was open and reachable. This points to connection limit exhaustion, resource overloading, or rate limiting on the target server rather than a routing or firewall block.

- The tight temporal coupling between the `v2.14.3` deployment (`14:31:13`) and the first failure (`14:32:40`), combined with the healthy `v2.14.2` control period, makes the deployment the most plausible trigger for the routing change.
- The metrics uploads are unrelated because they start hours before the checkout issues, run on a separate thread, and target a different port (`9092`).

Possible Root Cause

Release `v2.14.3` (commit `219c39f`), deployed to production at `14:31:13.000`, introduced or activated a routing change that directed odd user ID checkouts to the upstream instance at `10.0.3.44:8443`. That service was either misconfigured, lacked sufficient connection pool capacity, or suffered socket exhaustion under load, resulting in intermittent TCP resets. Even user checkouts remained routed to the stable legacy backend and did not experience issues.

Confidence Level

- **High (90%)** for the traffic routing partition (even vs. odd split) and target failure at `10.0.3.44:8443`.
- **High (85%)** that the `v2.14.3` deployment triggered the incident, given the 87-second gap between deployment and first failure and the healthy control period after `v2.14.2`. The logs establish the temporal link; confirming the exact mechanism (routing rule, feature flag, or configuration change in commit `219c39f`) would require the release diff.

Investigation Process

- **Parsed Logs:** Extracted log fields from `web.log` and `worker.log` using regular expressions, with explicit accounting for every line: 100,000 request lines, 2 deploy events, and 1,500 slow-query warnings in `web.log`; 27,917 job outcomes and 793 metrics errors in `worker.log`. No line was silently discarded.
- **Correlated Request IDs:** Mapped background worker executions to originating HTTP requests using the `request_id` field.
- **Audited Deployment History:** Found two `[deploy]` events in `web.log`. Release `v2.14.2` (`06:01:47`) was followed by hours of healthy checkouts and was ruled out; release `v2.14.3` (`14:31:13`) preceded the first failure by 87 seconds and is identified as the probable trigger (see Bonus).
- **Isolated Failure Path:** Audited request volumes and background task outcomes to identify `/checkout` as the sole failing route.
- **Identified Failure Boundary:** Traced the earliest worker failure to establish the transition timestamp of `14:32:40.073` for the incident.
- **Segmented by Parity:** Grouped post-incident transactions by user ID characteristics and identified that failures only affected odd user IDs.
- **Quantified Parity Metrics:** Calculated the 100% success rate for even users and the 87.01% failure rate for odd users.
- **Evaluated Durations:** Compared baseline durations of completed checkout tasks before and after the incident to check for performance degradation.
- **Traced Client Re-attempts:** Audited logs for users who submitted multiple checkout requests, confirming that the upstream resets were intermittent.
- **Ruled Out Metrics Warnings:** Checked the timing, thread identifiers (`[metrics-worker]`), and target host (`analytics.internal:9092`) of the metrics errors, identifying them as unrelated background noise.

- **Ruled Out Slow Database Queries:** `web.log` contains 1,500 `WARN [db] slow query` entries against the `products` (383), `carts` (396), `orders` (375), and `sessions` (346) tables, spanning the full day (`00:02:38` to `23:59:11`) with hourly counts that track overall traffic volume and show no step change at the incident boundary. They are pre-existing performance noise unrelated to the checkout failures.
- **Ruled Out Authentication Failures:** The 118 `401` responses on `/api/user` begin at `02:24:48`, hours before the incident, and follow the traffic curve. They are routine session expirations, not a cause or symptom of the checkout issue.
- **Ruled Out Expected 404 Responses:** The 192 `404` responses are confined to `/product/:id` lookups, begin at `03:54:29`, and show no correlation with the incident window. They reflect requests for nonexistent product IDs.
- **Counted Impacted Users:** Deduplicated the user IDs in the failed checkout records to confirm that 2,335 unique users were affected, with a failed-attempt distribution of 2,286 / 48 / 1 (one, two, and three failures respectively).